L2C Instruction-Aware RRIP: Task 2a and 2b Report

ChampSim, 1 core, 25M warmup + 25M simulated instructions, 3 traces. Only the L2C replacement policy changes; LLC and all other levels use LRU.
Baseline = LRU at L2C. **Task 2a** = static I-RRIP.
Task 2b = dynamic I-RRIP (leader stride 32, epoch 4096).

Speedup = $\text{IPC}_{\text{policy}}/\text{IPC}_{\text{baseline}}$. Load MPKI values come from the `L2C LOAD`, `L2C INSTRUCTION LOAD` and `L2C DATA LOAD` MPKI lines of each output file. Averages of speedup are geometric means.

1 Task 2a: Static I-RRIP

1.1 Rationale for the insertion, promotion and eviction decisions

The policy (`irrip.l2c_repl`) uses a 2-bit RRPV (maximum 3) and one extra bit per line recording whether the resident line is an instruction line. All three decisions are modified, each in a small way.

Decision	SRRIP-style behaviour	I-RRIP (Task 2a)
Insertion	Every demand fill at RRPV 2	Demand instruction fill: **1**. Prefetched instruction fill: 2. Demand data fill: 2. Prefetched data or writeback fill: **3**.
Promotion	Hit resets RRPV to 0	Hit lowers RRPV by **one step** (floor 0), for every line kind.
Eviction	First line at RRPV 3	Still needs RRPV 3. If an instruction and a data line both sit at 3, evict the **data line**, except 1 time in 4 (global counter) when the instruction line goes.

Table 1: Policy parameters of Task 2a.

1. Why the problem exists. Trace2 has an L1I MPKI of 40.0 and a baseline L2 instruction load MPKI of 18.9, more than twice its L2 data load MPKI (8.7). Under LRU, instruction, data and translation lines are treated alike, so streaming data fills evict hot code. In trace2 the baseline recorded 139,041 data-evicts-instruction events.

2. Insertion: instruction lines start one step closer to reuse. A demand instruction fill enters at RRPV 1 against 2 for demand data, so it needs one more round of aging before it becomes a victim. Instruction fetches have strong temporal and spatial locality and an instruction miss stalls the front end, so a small head start is cheap insurance. Prefetched data and writeback fills enter at 3 (immediately evictable) because they are the least likely to be re-read soon. The gaps are deliberately small, one RRPV step: a large gap would become hard protection and starve data.

3. Promotion: gradual, not jump-to-zero. A hit moves the line down by one step instead of straight to 0. One hit is weak evidence of reuse, so a line has to earn full protection by repeated hits. A single stray touch cannot pin a line. It applies equally to instruction and data lines, which keeps the policy simple. `PROMOTE_ON_PF_WB_HIT` is 1, so prefetch and writeback hits also promote.

4. Eviction: RRPV decides, instruction bias only breaks ties. The victim must be at RRPV 3, exactly as in RRIP. The bias applies only when instruction and data lines are both at 3, i.e. both predicted dead. Data is then evicted by default, but one time in four (`INSTR_EVICT_PERIOD = 4`) the instruction line is evicted. Instruction priority therefore never becomes permanent: dead code is eventually cleared and a data-hot phase is never fully blocked. The counter is a global deterministic counter rather than `rand()`, so runs are reproducible.

5. Why this is the best static choice here.

- Largest gain where it matters: +1.97% IPC on trace2, with instruction MPKI down 23.9%. Instruction-evicts-instruction events fall 35.3%.
- Bounded cost: data load MPKI rises 1.2%–3.4% as data lines lose some capacity. Data-evicts-data events rise 15.1% on trace2.
- Trace1 gains only +0.08%: it is limited by branch accuracy (91.2%) and a 14.7% BTB miss rate, so the saved L2 instruction misses were not on the critical path. Trace3 has instruction MPKI of 1.1, so nothing is gained and nothing is lost (−0.006%).
- Cheap: one bit per line plus one global counter.
- The tie-break period is tunable. A larger period protects code harder and costs data more.

Limitation: the data-MPKI increase is unconditional in the static design, which motivates Task 2b.

Check against the assignment goal (reduce L2 load instruction MPKI): achieved on all three traces: 10.37 to 10.09 (trace1), 18.89 to 14.38 (trace2), 1.12 to 1.08 (trace3), average 10.13 to 8.52.

1.2 Results

Trace	Policy	IPC	Speedup	Load MPKI	Instr MPKI	Data MPKI
trace1	Baseline (LRU)	0.232246	1.0000	17.7027	10.3682	7.3345
trace1	I-RRIP	0.232440	1.0008	17.6615	10.0929	7.5686
trace2	Baseline (LRU)	0.251304	1.0000	27.5596	18.8918	8.6677
trace2	I-RRIP	0.256243	1.0197	23.3356	14.3775	8.9580
trace3	Baseline (LRU)	0.381855	1.0000	3.9603	1.1237	2.8366
trace3	I-RRIP	0.381833	0.9999	3.9505	1.0805	2.8700
Average	Baseline (LRU)	0.288468	1.0000	16.4075	10.1279	6.2796
Average	I-RRIP	0.290172	1.0068	14.9825	8.5170	6.4656

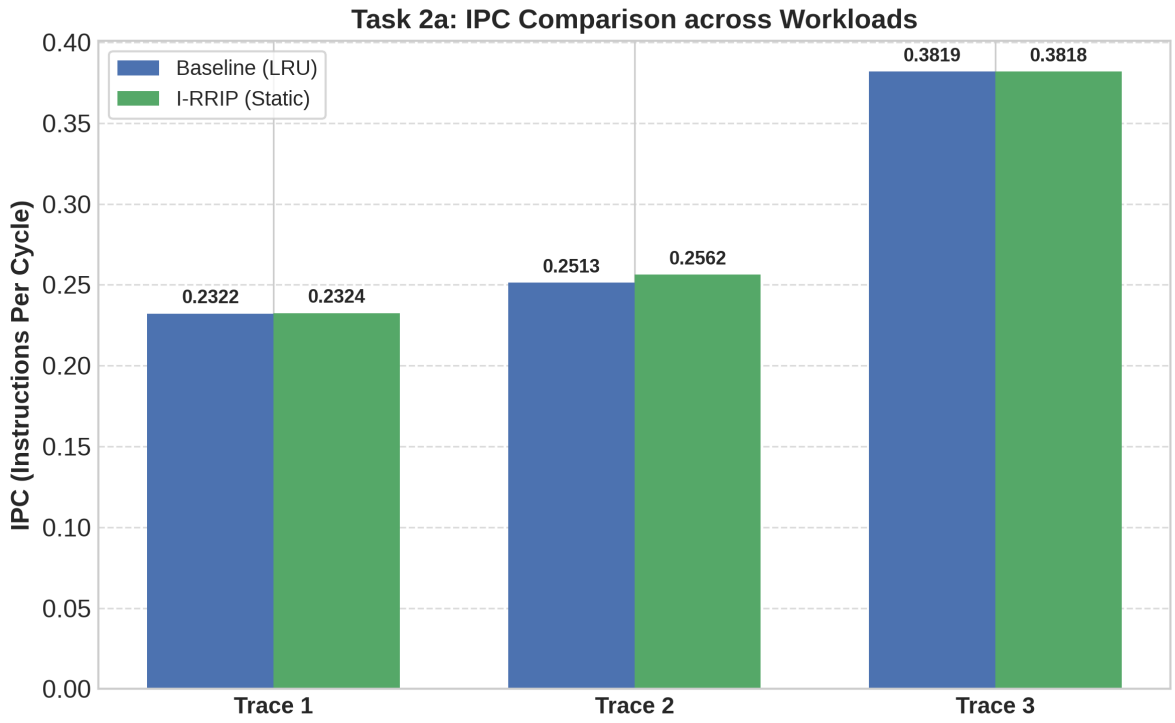
Table 2: Baseline vs static I-RRIP. Load MPKI = Instr MPKI + Data MPKI.

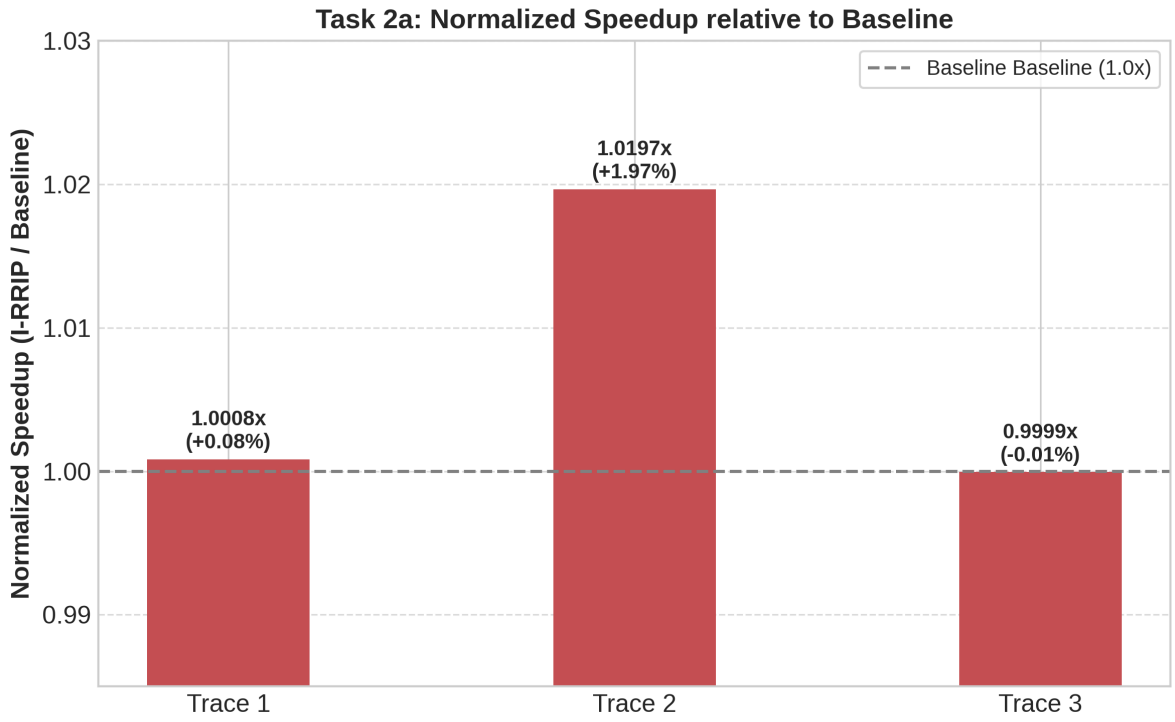
1.3 What changed in the instruction-bound trace (trace2)

Metric	Baseline	I-RRIP	Change
L2 instruction load MPKI	18.89	14.38	−23.9%
L2 data load MPKI	8.67	8.96	+3.3%
L2C total load MPKI	27.56	23.34	−15.3%
Instructions evicting instructions	295,827	191,336	−35.3%
Data evicting data	143,202	164,806	+15.1%

Table 3: Eviction-pair counters from the L2C statistics, trace2.

1.4 Plots





2 Task 2b: Dynamic I-RRIP

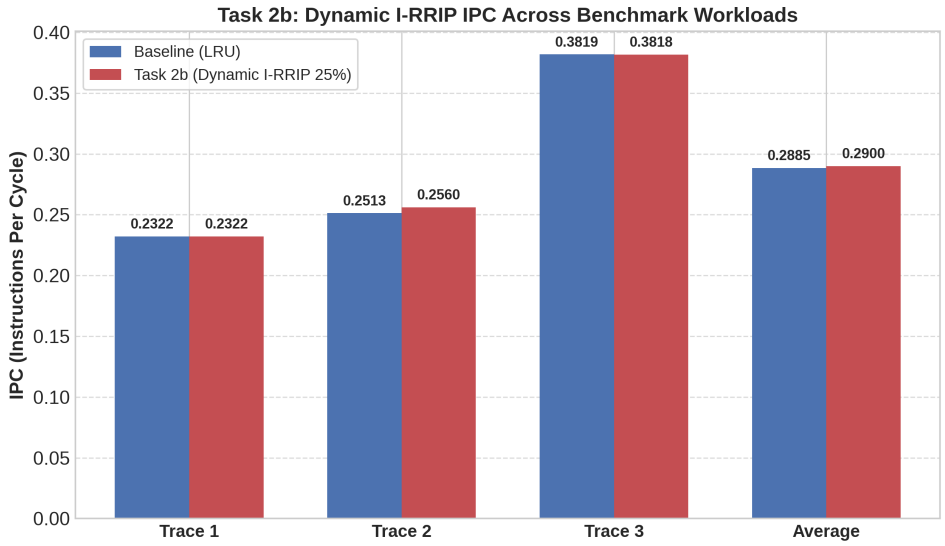
2.1 Results (threshold 25%, leader stride 32, epoch 4096)

Trace	Policy	IPC	Speedup	Load MPKI	Instr MPKI	Data MPKI
trace1	Dynamic I-RRIP	0.232161	0.9996	17.6584	10.1144	7.5441
trace2	Dynamic I-RRIP	0.256050	1.0189	23.3820	14.4892	8.8928
trace3	Dynamic I-RRIP	0.381800	0.9999	3.9560	1.0860	2.8700
Average	Dynamic I-RRIP	0.290004	1.0061	14.9988	8.5632	6.4356

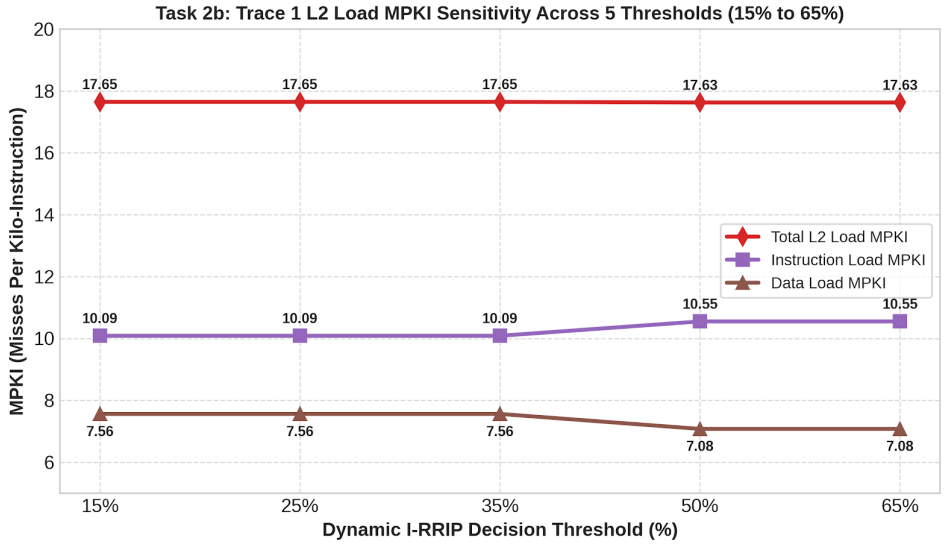
Table 4: Dynamic I-RRIP results.

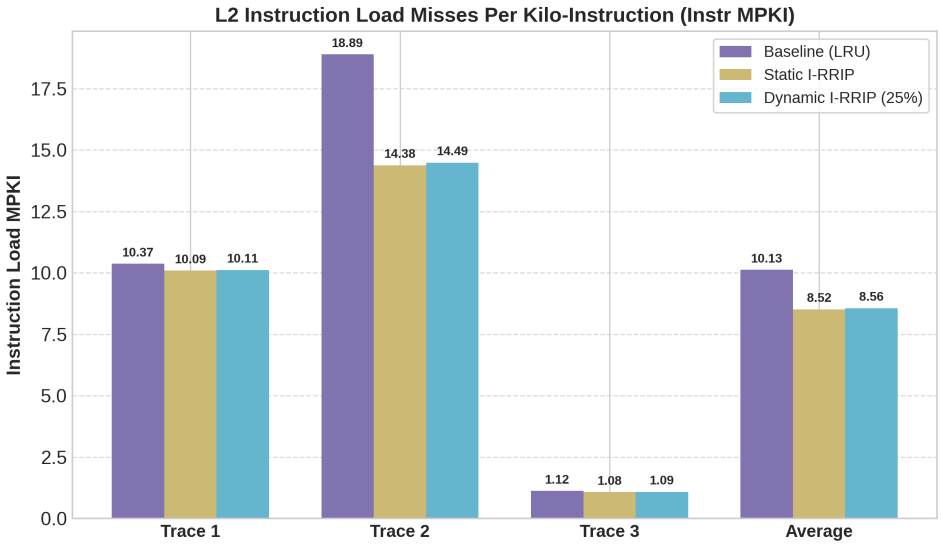
Policy	Avg IPC	Geomean speedup	Load MPKI	Instr MPKI	Data MPKI
Baseline LRU	0.288468	1.0000	16.4075	10.1279	6.2796
Static I-RRIP	0.290172	1.0068	14.9825	8.5170	6.4656
Dynamic I-RRIP	0.290004	1.0061	14.9988	8.5632	6.4356

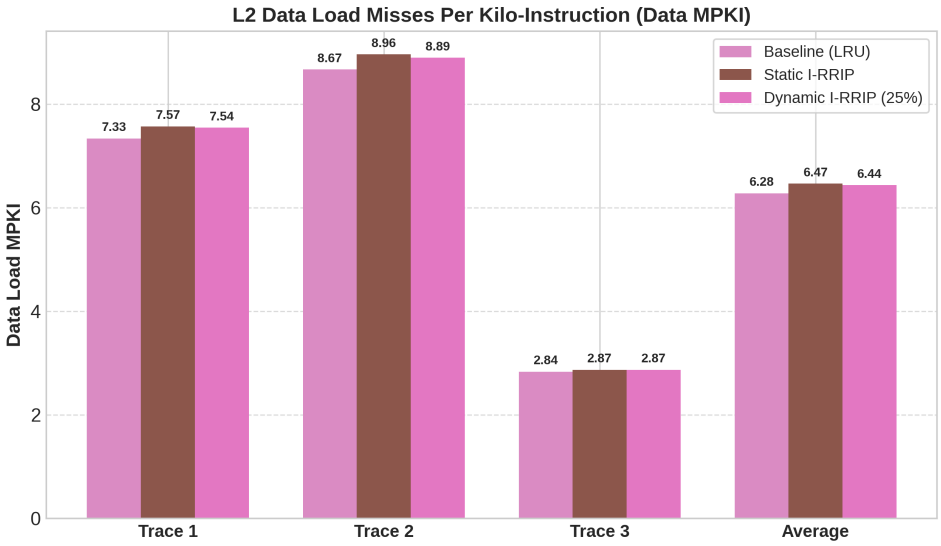
Table 5: Averages over the three traces.

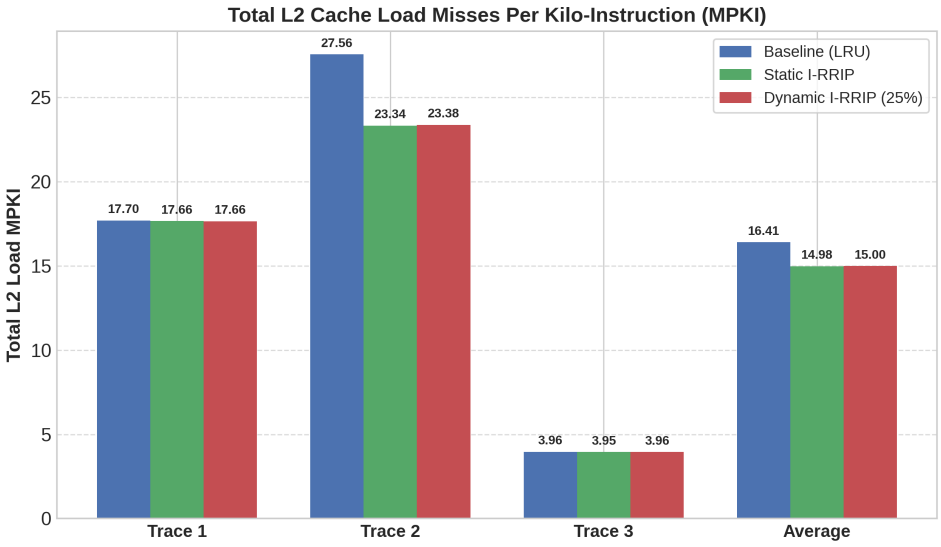

Task 2b: Dynamic I-RRIP IPC Across Benchmark Workloads
Baseline (LRU)
Task 2b (Dynamic I-RRIP 25%)
IPC (Instructions Per Cycle)
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0.2322
0.2322
0.2513
0.2560
0.3819
0.3818
0.2885
0.2900
Trace 1
Trace 2
Trace 3
Average

2.2 Plots


Task 2b: Trace 1 L2 Load MPKI Sensitivity Across 5 Thresholds (15% to 65%)
MPKI (Misses Per Kilo-Instruction)
20
18
16
14
12
10
8
6
Total L2 Load MPKI
Instruction Load MPKI
Data Load MPKI
17.65
17.65
17.65
17.63
17.63
10.09
10.09
10.09
10.55
10.55
7.56
7.56
7.56
7.08
7.08
15%
25%
35%
50%
65%
Dynamic I-RRIP Decision Threshold (%)


L2 Instruction Load Misses Per Kilo-Instruction (Instr MPKI)
Instruction Load MPKI
Baseline (LRU)
Static I-RRIP
Dynamic I-RRIP (25%)
10.37
10.09
10.11
18.89
14.38
14.49
1.12
1.08
1.09
10.13
8.52
8.56
Trace 1
Trace 2
Trace 3
Average


L2 Data Load Misses Per Kilo-Instruction (Data MPKI)
Data Load MPKI
Baseline (LRU)
Static I-RRIP
Dynamic I-RRIP (25%)
7.33
7.57
7.54
8.67
8.96
8.89
2.84
2.87
2.87
6.28
6.47
6.44
Trace 1
Trace 2
Trace 3
Average


Total L2 Cache Load Misses Per Kilo-Instruction (MPKI)
Total L2 Load MPKI
Baseline (LRU)
Static I-RRIP
Dynamic I-RRIP (25%)
17.70
17.66
17.66
27.56
23.34
23.38
3.96
3.95
3.96
16.41
14.98
15.00
Trace 1
Trace 2
Trace 3
Average

2.3 Sensitivity study of the threshold

The sweep covers thresholds of 15%, 25%, 35%, 50% and 65% on all three traces (15 runs). Speedup is IPC over the LRU baseline; the last three rows of each table give the LRU baseline, Task 2a and the original Task 2b run for reference.

Threshold	IPC t1	IPC t2	IPC t3	Avg IPC	Sp. t1	Sp. t2	Sp. t3	Geomean
15%	0.232247	0.255522	0.382031	0.289933	1.0000	1.0168	1.0005	1.0057
25%	0.232247	0.255522	0.382031	0.289933	1.0000	1.0168	1.0005	1.0057
35%	0.232247	0.255201	0.382031	0.289826	1.0000	1.0155	1.0005	1.0053
50%	0.232206	0.254380	0.381726	0.289437	0.9998	1.0122	0.9997	1.0039
65%	0.232206	0.252063	0.381726	0.288665	0.9998	1.0030	0.9997	1.0008
LRU baseline	0.232246	0.251304	0.381855	0.288468	1.0000	1.0000	1.0000	1.0000
Task 2a (static)	0.232440	0.256243	0.381833	0.290172	1.0008	1.0197	0.9999	1.0068

Table 6: IPC and speedup versus threshold.

Threshold	Avg Load MPKI	Avg Instr MPKI	Avg Data MPKI
15%	15.0835	8.6618	6.4216
25%	15.0835	8.6618	6.4216
35%	15.1277	8.7302	6.3976
50%	15.3291	9.1320	6.1971
65%	15.7624	9.7288	6.0337
LRU baseline	16.4075	10.1279	6.2796
Task 2a (static)	14.9825	8.5170	6.4656

Table 7: L2C load MPKI versus threshold (averages over the three traces).

What the sweep shows.

- **Speedup falls as T rises.** Geomean speedup is 1.0057 at 15% and 25% (identical runs: every trace stays in I-RRIP mode in every epoch), 1.0053 at 35%, 1.0039 at 50% and 1.0008 at 65%. The loss is almost all trace2, whose speedup drops from 1.0168 to 1.0030.
- **The trade is visible in MPKI.** From 25% to 65%, average instruction MPKI rises from 8.66 to 9.73 and data MPKI falls from 6.42 to 6.03. A higher threshold switches protection off, giving back instruction hits for data hits.
- **Mode counts show where each trace flips.** Trace3 is on at 25%, 5 of 6 epochs at 35% and off at 50%. Trace1 is on at 35% and off at 50%. Trace2 sits near the boundary at 50%–65% (14 of 19 epochs on at 50%, 3 of 19 at 65%). So the leader-measured instruction share is roughly 25–35% for trace3, 35–50% for trace1 and 50–65% for trace2. This corrects my earlier estimate for trace3 (22%).
- **No threshold beats static I-RRIP** (1.0068). The best dynamic setting reaches 1.0057. On trace3 the dynamic run is slightly ahead of static (1.0005 against 0.9999), but 0.05% is within the noise of these runs.
- **Baseline mismatch is small.** At 50% and above, trace1 and trace3 run in SRRIP mode for almost the whole run. Their IPC (0.232206 and 0.381726) is only 0.02% and 0.03% below LRU, so the SRRIP-versus-LRU difference barely affects the other results.

2.4 Rationale for the leader sets and decision logic

The design (`dyn_irrip.l2c_repl`) follows the assignment's set-dueling scheme. Leader sets always run plain RRIP and collect statistics; the follower sets run I-RRIP or RRIP depending on a formula over those statistics. The leaders do not compare two policies' hit counts: they measure **how much of the demand traffic is instruction traffic**, and the formula turns that share into a mode.

Leader sets: every 32nd set. With 1024 L2C sets that is 32 leaders, about 3.1% of the cache. The other 992 follower sets run I-RRIP when the mode is on and SRRIP when it is off.

- **Why leaders always run SRRIP.** The statistic they gather should not depend on the mode being decided. If leaders ran I-RRIP while on, the policy would change the traffic it is measuring and the decision could chase itself.
- **Why 32 and not fewer.** Few leaders give a noisy sample. 32 sets spread evenly over the index space cover many address regions.
- **Why not many more.** Leaders are forced into SRRIP even when I-RRIP is winning, so each extra leader costs a little of the gain. At 3.1% at most about 3% of sets are forced off the better policy.
- **Why a regular stride.** `set % 32 == 0` is a single compare, with no tag or table.

Decision logic. Four counters are kept: instruction and total demand accesses, and instruction and total demand misses. Writebacks are ignored because they are not demand traffic. At the end of each epoch:

$$(i_{acc} + i_{miss}) \times 100 \;>\; T \times (t_{acc} + t_{miss}) \;\Rightarrow\; \text{I-RRIP on, else off.}$$

Counting misses in addition to accesses weights instruction misses double, matching their cost, since an instruction miss stalls the front end. Only an integer multiply and compare are needed.

Epoch: 4096 leader events. Long enough to gather thousands of samples per decision, so the mode does not flip on noise; short enough to follow phase changes. After each decision the counters are *halved* instead of cleared, so the previous phase fades out gradually, giving built-in hysteresis. Before the first decision the mode is I-RRIP (`INIT_IRRIP_MODE = 1`), the safe default because Task 2a rarely hurts.

What the sweep measured for the three traces. At $T = 25\%$ every trace stayed in I-RRIP mode for every epoch (11 of 11, 19 of 19, 6 of 6), so Task 2b at 25% behaves like Task 2a. That is why trace2 keeps nearly all of its static gain (+1.68% against +1.97%) and why traces 1 and 3 are flat. The epoch counts at higher thresholds locate each trace's instruction share between about 25% and 65%.

2.5 Does dynamic beat static?

Not on these three traces.

- Dynamic is slightly behind static on every trace (1.0189 vs 1.0197 on trace2; 0.9996 vs 1.0008 on trace1); the geomean gap is 0.07 percentage points.
- It reduces the cost: the trace2 data-MPKI penalty shrinks from +3.3% to +2.6%, and trace1 from +3.2% to +2.9%.

- It gives back some instruction benefit (trace2 instruction MPKI 14.49 vs 14.38), most likely because of the 32 SRRIP leader sets and any epochs spent in SRRIP mode.

Dynamic is the right design for data-dominated workloads where static I-RRIP would cost real IPC; none of these traces is like that (worst static result: -0.006%). Its case here rests on robustness, not measured gain. On these traces static wins by a very small margin. The IPC differences on traces 1 and 3 ($\sim$0.0001–0.0002) are small; only trace2 shows a clearly real effect.

Baseline mismatch. Every output file reports an LRU baseline at L2C, while the SRRIP mode in `dyn_irrip.l2c_repl` is plain SRRIP. In Task 2b the leaders, and followers during any SRRIP epoch, therefore differ from the baseline by the SRRIP-versus-LRU effect as well as by I-RRIP. For a clean attribution, add an SRRIP-only run on the same three traces.